MuniPricer Process Evidence Packet

Anubhav Awasthi, Ibukun Agbaje and Eno Okon · BUS 43100 AI Essentials · November 4, 2026

MuniPricer estimates a fair price for a municipal bond that has not traded recently, with an 80% range and a plain-English explanation of what moved the number. This packet shows how the project got from a broad idea in week 1 to a deployed v2 in week 5, including the dead ends.

Work in progress

What	Where	Notes
Live app (Streamlit)	munipricer.streamlit.app	CUSIP lookup or manual entry, buy/sell side, trade size, 80% range, factor chart
Code and version history	github.com/anub-awasthi/munipricer	10 commits, tests run on every push
Evaluation report	reports/evaluation.md	Rolling backtest, ablations, error analysis, charts
Web demo (no install)	MuniPricer demo	Precomputed predictions for 2,731 bonds

Contents. (1) An annotated trail of our two main Claude conversations. (2) A failure exhibit. (3) Our weekly diary, with the GitHub commit log as version history. (4) A before-and-after comparison of v1 and v2, with the checks we ran.

All accuracy numbers below are on simulated trade data. We now have WRDS access, and retraining on real MSRB trades is the first task of week 7.

1. Annotated conversation trail

Six turning points across two Claude conversations, in order. Each one says what Claude proposed, what we did, and why. Quotes are copied from the transcripts.

- **Conversation A**, “AI essentials class” (Claude chat, Oct 21 to 28): scoping, competitor scan, v1 build. Open conversation A
- **Conversation B**, MuniPricer build-out (Claude Code session, Nov 4): v2 model, evaluation, deployment, WRDS. Open conversation B

Turning point 1: we dropped half the idea (Conversation A, Oct 21)

What happened. Our week 1 idea had two halves: an issuance assistant for municipalities and a secondary-market pricing tool. Claude asked which half we expected to build by week 10.

Our call. Pricing only. Working through the Lab 3 scaffold, the issuance side kept failing two questions: “what decision changes” and “what data do we have rights to.” Offering documents are unstructured PDFs and the users are hard to reach in ten weeks.

Turning point 2: the competitor scan changed who we build for (Conversation A, Oct 21)

What happened. The scan found three firms already doing ML muni pricing at scale: SOLVE Px, Tradeweb Ai-Price and Spline Data. Claude’s critique ended: “the model is not the product. The audience and the explanation layer are.” It also warned: “the competitor essay commits you to a ‘small holder lookup with explanations’ angle. If IBK and Eno see the product differently, settle that before you submit.”

Our call. We kept the small-holder angle. As our week 3 journal put it, “Our angle has to be access for small holders rather than beating Bloomberg-tier vendors on accuracy.” The incumbents sell data feeds to dealers and asset managers. None of them serves a holder with one CUSIP, and none explains a price in plain English.

Turning point 3: we overruled Claude’s recommended setup (Conversation A, Oct 27)

What happened. Claude reported “I can’t reach MSRB EMMA or any muni data source from here” and offered options. It recommended synthetic data first, a “Python app you run locally (Recommended)”, and left scope open.

Our call. We took synthetic data first. We rejected the local-only recommendation and chose both a Streamlit app and a shareable web demo. A local Python app would only reach people who can install Python. A link reaches classmates, TAs and the small holders we are building for. We chose the full scope, prediction plus range plus explanation, because the competitor scan found that none of the incumbents explain a price in language a non-trader can read.

Turning point 4: we kept v1 as the baseline (Conversation B, Nov 4)

What happened. Our GitHub repo was empty and the v1 code only existed as a zip. Claude offered to rebuild everything from our handoff doc or to start from the zip.

Our call. We uploaded the zip so v1 became the first commit. Every v2 change since then is measured against it, which is what makes the before-and-after in section 4 possible.

Turning point 5: Claude found two problems we had shipped (Conversation B, Nov 4)

What happened. Claude reported: “I changed the model, not just the plumbing.” v1’s errors drifted over the test window (section 2). It also found our buy/sell labels were backwards: “In MSRB data ‘P’ is a dealer purchase from a customer (you sold) and ‘S’ is a dealer sale to a customer (you bought).” Our v1 app and web demo had shown a markdown where a buyer should see a markup.

Our call. We accepted the spread target after reading the ablation table, which shows the v1 approach next to v2 on the same data. We checked the buy/sell claim against MSRB’s own field definitions, which confirm it (source). We then added a test that a buyer always pays more than a seller.

Turning point 6: a real deployment exposed a design flaw (Conversation B, Nov 4)

What happened. Anu asked “Is the live website active?” The only thing live was the precomputed web demo. So we deployed the real app to Streamlit Community Cloud. The first load sat on “Training LightGBM models” for over five minutes, and Anu flagged it: “its still going, its taken 5 minutes.”

What we learned. The app trained itself on first launch, which worked in testing. Free Streamlit apps sleep and lose their files, so every visitor after a quiet period would have waited five minutes. No test caught this; using the product did. We now ship the trained model with the code, and the app loads in seconds.

2. Failure exhibit: we hit the same failure twice before we understood it

Tree models cannot predict values outside the range they were trained on. We learned this once with a feature, missed that it applied to our target too, and only caught it a week later.

First time (Oct 27, v1). Our first model included a calendar feature: days since the start of the data. Test trades come after all training trades, so every test value was beyond anything the trees had seen. It quietly made test results worse, so we removed it.

The fix that hid the problem (Oct 27, v1). The model’s 80% price range only contained 61% of real test trades. We added a calibration step that widens the range using held-out trades, which brought coverage to 77%. We called that acceptable for v1. In hindsight the calibration was set to aim for 85%, which papered over the gap instead of explaining it.

Second time (Nov 4, v2). Looking at errors week by week, the model’s yield predictions drifted further off as the test window went on. The average miss grew from -0.05 to -0.12 yield points over eight weeks. The cause was the same as the calendar feature. The model predicted absolute yield, and rates moved during the test window to levels the trees had not seen. Recalibrating honestly at 80% dropped v1’s coverage to 71%, which confirmed the drift was real.

What we tried and what worked. We changed what the model predicts. Instead of yield, it now predicts the bond’s spread over a trailing 3-day market average, and the market average carries the rate level. On the same test data, price error fell from 0.85 to 0.64 points per 100, and the 80% range covered 79.4% without any fudge.

What we concluded. A tree model should predict the part of the answer that stays stable over time and let something else carry the part that moves. A calibration step can make a broken model look fine, so check errors over time and not only the overall average.

A second, smaller failure: our own test data fooled us

Our ablation test removes groups of features to see what each is worth. On v1’s synthetic data, removing the bond’s own trade history made the model slightly better (error 0.643 with it, 0.637 without). That contradicts the premise of the whole project: in the real market, a bond’s last trade is the best clue to its price.

The problem was our simulator, not the model. It gave every bond about 20 trades a year and only 12 basis points of bond-specific spread, so the model could identify each bond from its rating and size alone. We made the simulated data harder: 35 basis points of bond-specific spread, and trading that is concentrated,

with 32% of bonds trading fewer than five times a year. Now removing trade history costs what it should (0.71 to 0.81).

The lesson: data you generate yourself can confirm whatever you built into it. That is the main reason real MSRB data is our first task for week 7.

3. Weekly diary

We kept the diary as weekly entries in a Claude project for this class, starting in week 3. Weeks 1 to 4 are copied from the homework entries already submitted, shortened in places but not reworded. From week 5 the code is in GitHub, so the commit log below is the version history.

Weeks 1 to 3 (submitted with the week 3 homework, Oct 21)

- Weeks 1 and 2: We started with the brainstorming scaffold from Lab 3 and used it to pressure-test a few ideas before landing on the municipal bond market. The pain is clear: most munis trade rarely, so there is no reliable current price for the bond an investor or treasurer is holding. Our team is IBK, Eno and me.
- The original idea was broad. We wanted both an issuance assistant for municipalities and a secondary-market pricing and liquidity tool. In working through the scaffold, the issuance side kept failing the “what decision changes” and “what data do we have rights to” questions. Offering documents are unstructured PDFs and the users are hard to reach for a ten-week project.
- Week 3: We narrowed the build to the secondary-market pricing model. The AI output is a prediction: given a bond’s features and recent trades in similar bonds, what is a fair price or yield today? This maps cleanly to the supervised learning material from week 2 and gives us a measurable model metric (error against the next actual trade).
- Week 3: I did a competitor scan this week and found SOLVE Px, Tradeweb Ai-Price and Spline Data. All three already do machine-learning muni pricing at scale. That was deflating for about an hour. The upside is that it confirmed the problem is real and gave me a benchmark for what features matter. Our angle has to be access for small holders rather than beating Bloomberg-tier vendors on accuracy.
- What has not worked yet: nothing has failed because nothing has been built. The honest gap is data. I have not pulled anything from MSRB EMMA yet and I don’t know how much historical trade data is downloadable for free versus paywalled. That is my first task for week 4 and the most likely place we pivot.
- Tools so far: I used Claude for research and scoping. Planned: a Python notebook for the model and a vibe-coding tool for a simple front end where a user enters a CUSIP and gets a predicted price with a confidence range.

Week 4 (submitted with the week 4 homework)

This week I built the first working version of our tool, which we’re calling MuniPricer. The goal is to estimate a fair price for a municipal bond that hasn’t traded recently, with a confidence range and a plain-English explanation of what drove the number.

The biggest decision was what to do about data. MSRB trade data isn’t freely downloadable in bulk, and I learned this week that Booth has access to it through WRDS. I’ve requested access, but rather than wait, I built a realistic synthetic dataset (about 60,000 trades across 3,000 bonds) that mimics how munis actually trade. The whole pipeline is written so a real WRDS export drops in with one command.

On the model side, I started with a linear regression baseline and then trained gradient-boosted trees (LightGBM). On a held-out test window of the most recent 60 days, the boosted model's average price error was 0.85 points versus 1.40 for the baseline. Two things didn't work the first time. The 10th to 90th percentile range only captured 61% of real trades, so I added a calibration step, which brought it to 77%. I also had to drop a "trade date" feature because tree models can't extrapolate into the future.

Week 5 (Nov 4)

- Moved the code from a zip into GitHub with v1 as the first commit, so every later change is measured against it. Added automated tests that run on every push.
- Changed what the model predicts, from yield to spread over a 3-day market average, after finding v1's errors drifted over time (section 2). Same test data: price error 0.85 to 0.64, honest range coverage 79.4%.
- Found that our v1 app labeled buy and sell backwards relative to the MSRB convention. Fixed in the app, the web demo and the docs.
- Built an evaluation harness. Across four separate 30-day test windows, error is 0.71 ± 0.02 points and coverage 79.4% ± 0.4%, so one lucky window doesn't explain the result.
- Our ablation test showed our simulated data was too easy. Rebuilt the simulator so bond history matters as it does in the real market.
- Compared against how a small holder prices a bond today, by looking up its last trade: 1.41 points of error for that method vs 0.70 for the model (section 4).
- Got WRDS access. Built a loader that cleans a raw MSRB export and a script that pulls trades from WRDS directly.
- Deployed the app publicly on Streamlit. The first load took over five minutes, which exposed a design flaw. Fixed the same day.
- What didn't work: the cold-start design, the easy simulator, the buy/sell labels. What we still haven't done: test a single prediction against a real trade.

Commit log (version history)

Time (CT)	Commit	What changed
Nov 4, 17:30	e640003	Script to pull MSRB trades from WRDS
Nov 4, 17:24	d29b5a4	Ship the trained model with the code so the live app loads instantly
Nov 4, 17:16	e4d707d	Dev container config added from Streamlit
Nov 4, 16:08	478daef	README for v2
Nov 4, 16:02	5517e02	Fix a failing app test
Nov 4, 15:52	4159dc9	App and web demo updated for the spread model; optional Claude rewrite of explanations
Nov 4, 15:34	78026e0	Evaluation harness (backtest, ablations, error report); harder simulated data
Nov 4, 15:17	3a0dd08	Loader for raw WRDS/MSRB exports

Time (CT)	Commit	What changed
Nov 4, 15:14	f228312	Spread target, per-staleness range calibration, tests
Nov 4, 15:03	6ee2666	v1 imported from the Oct 27 zip

4. Before and after, and how we checked it

v2 cut price error by a quarter on the same data, made the range honest, and fixed a labeling bug v1 shipped with.

	v1 (Oct 27)	v2 (Nov 4)
What the model predicts	Yield	Spread over a 3-day market average
Price error on v1's test data (points per 100)	0.85	0.64
80% range: share of real trades inside	77%, by aiming calibration at 85%	79.4%, aiming at 80%
Range width	One margin for every bond	Wider for bonds with older or no trade history
Buy and sell labels	Backwards vs MSRB convention	MSRB convention
Evaluation	One test window	Four-window backtest, ablations, error report
Real-data path	Rename columns by hand	WRDS pull script and a loader with a cleaning report
Delivery	Local app, web demo	Adds a public live app and tests on every push

What good looks like, and what we found

1. It must beat how a small holder prices a bond today. Today that means looking up the bond's last trade on EMMA. On the 9,926 test trades where a bond had traded before, that method misses by 1.41 points on average. The model misses by 0.70. Part of the model's edge is that it knows which side of the trade you are on, which a careful holder could adjust for by hand.

Time since the bond last traded	Test trades	Last-trade lookup error	Model error
Under 1 week	6,363	1.36	0.63
1 to 4 weeks	2,345	1.43	0.78
1 to 3 months	858	1.53	0.87
3 to 12 months	360	1.81	1.02

Errors are average absolute price error in points per 100 of face value.

2. The range must mean what it says. An 80% range should contain about 80% of real trades. Across four separate test windows it contained 79.4% ± 0.4%. By staleness bucket it runs from 78% to 83%.

3. Explanations must come from the model, not be invented. Every sentence in an explanation is built from the model's own feature contributions. The optional Claude rewrite is thrown away if any number in it differs from the original.

4. It must behave like the market. Automated tests check that a buyer pays more than a seller, that a bond priced cheap at its last trade is priced lower today, and that no feature uses a trade from the future. The buyer-pays-more test is what exposed the buy/sell bug in v1.

Problems we found. Bonds with no trade history in our data still miss by 2.76 points, and their range is wide to match. Everything above is on simulated data. We have not yet compared a single prediction with a real trade, and no one who trades munis has looked at the output. Retraining on real MSRB data and putting the tool in front of a muni practitioner are the first two tasks for weeks 7 and 8.